
ハッカソン ～計画書と設計書～

チーム 40

24G1021 : 宇井 祐真

24G1026 : 梅野 大誠

24G1136 : 山口 侑希

25G1020 : 薄井 悠希

25G1041 : 菊池 侑人

25G1059 : 佐藤 涼菜

目次

第 1 章	プロジェクトの計画書	1
1.1	プロジェクトの概要	1
1.1.1	プロジェクトの題目	1
1.1.2	プロジェクトの目的	1
1.1.3	既存の技術とプロジェクトの独自性	1
1.2	スコープ・マネジメント	2
1.2.1	成果物	2
1.2.2	プロジェクトの対象範囲と非対象範囲	2
1.2.3	機能分解構造 FBS	2
1.2.4	製品分解構造 PBS	2
1.2.5	FBS と PBS の対応関係	2
1.3	作業計画	2
1.3.1	作業分解構造 WBS と管理番号	2
1.3.2	アローダイアグラムとクリティカルパス	2
1.3.3	役割分担	2
1.3.4	ガントチャート	2
1.4	検証と妥当性確認: V&V 計画	2
1.4.1	プロジェクトの成果物に対する有効指標 MOE	2
1.4.2	有効指標 MOE に対応する性能指標 MOP	2
1.4.3	システムテストで評価する性能指標 MOP	2
1.4.4	結合テストで評価する性能指標 MOP	2
1.4.5	単体テストで評価する性能指標 MOP	2
1.4.6	プロジェクト中に監視する技術性能指標 TPM	2
1.4.7	プロジェクト中に監視する指標 TPM	2
1.5	資源・調達管理	2
1.6	リスク管理	2
第 2 章	システムの基本設計	3
2.1	機能一覧	3
2.2	システム構成図	3
2.3	操作インターフェース設計	3
2.4	状態遷移図	6

第 3 章	システムの詳細設計	7
3.1	部品一覧	7
3.2	ハードウェア設計	7
3.3	ソフトウェア設計	7
3.3.1	ファイル構成	7
3.3.2	オブジェクト設計 (クラス図)	8
3.3.3	関数設計	8
3.4	処理フローの設計	8
3.5	テストの設計	8

第 1 章 プロジェクトの計画書

1.1 プロジェクトの概要

1.1.1 プロジェクトの題目

1.1.2 プロジェクトの目的

プロジェクトの目的と最終的なゴール，解決すべき課題を記述する.

1.1.3 既存の技術とプロジェクトの独自性

1.2 スコープ・マネジメント

1.2.1 成果物

1.2.2 プロジェクトの対象範囲と非対象範囲

1.2.3 機能分解構造 FBS

1.2.4 製品分解構造 PBS

1.2.5 FBS と PBS の対応関係

1.3 作業計画

1.3.1 作業分解構造 WBS と管理番号

1.3.2 アローダイアグラムとクリティカルパス

1.3.3 役割分担

1.3.4 ガントチャート

1.4 検証と妥当性確認: V&V 計画

1.4.1 プロジェクトの成果物に対する有効指標 MOE

1.4.2 有効指標 MOE に対応する性能指標 MOP

1.4.3 システムテストで評価する性能指標 MOP

1.4.4 結合テストで評価する性能指標 MOP

1.4.5 単体テストで評価する性能指標 MOP

第 2 章 システムの基本設計

指揮デバイスについて説明を行う。

2.1 機能一覧

- ・無線通信によるデータ転送: 指揮デバイスを上下に振ることによってドラム (Arduino) に BPM 情報を送信.
- ・楽曲の音量制御: つまみを動かすことで楽曲の音量を調整.

2.2 システム構成図

指揮デバイスによる BPM 情報の送信方法を図 2.1 に, 指揮デバイスによる音量制御方法を図 2.2 に示す.

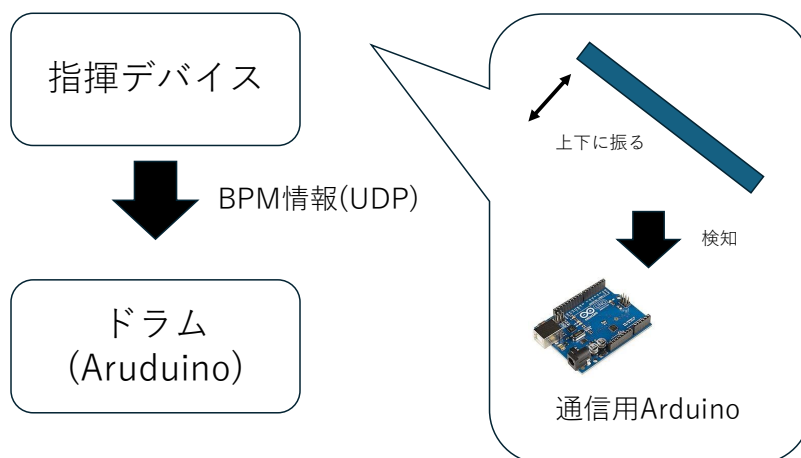


図 2.1 指揮デバイスとドラム Arduino の接続

2.3 操作インターフェース設計

指揮デバイスの概要図を図 2.3 に示す。また、指揮デバイスの操作に関する定義を表 2.1 に示す。

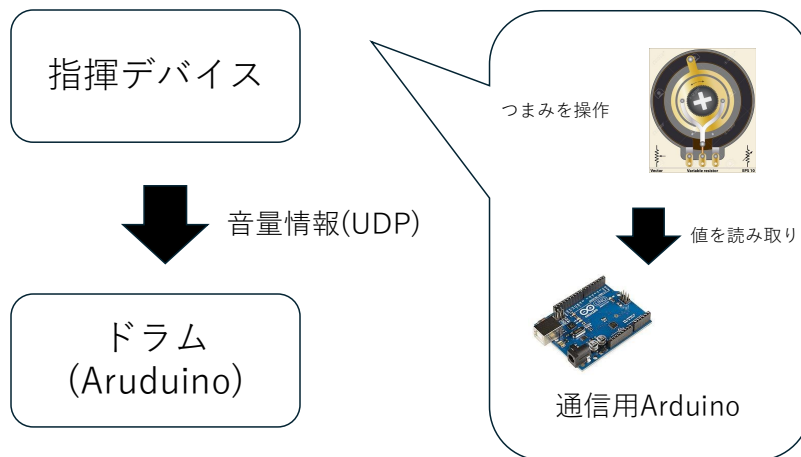


図 2.2 指揮デバイスによる音量調整

表 2.1 指揮デバイス 操作定義一覧

操作対象	ユーザーのアクション	システムの挙動
加速度センサ	デバイスを上下に振る	動きを検知し，設定された閾値を超えた瞬間に楽曲の BPM 情報をドラム側へ送信.
可変抵抗器 (つまみ)	つまみを左右に回して絞りを調整する	アナログ値を読み取り，BPM 送信とは独立して音量情報をリアルタイムに送信.

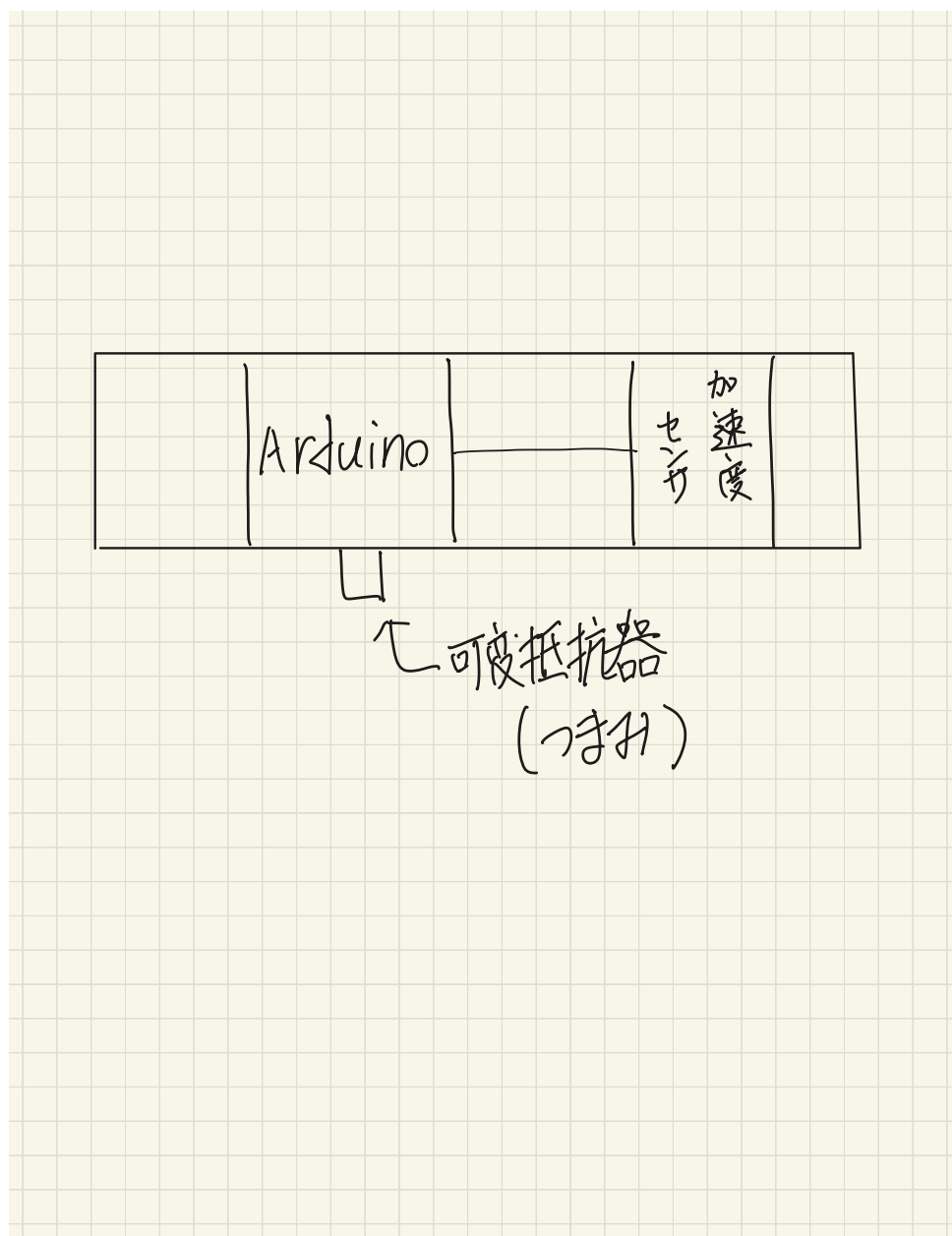


図 2.3 指揮デバイスの概要図

2.4 状態遷移図

指揮デバイスの内部で処理される状態遷移について図 2.4 に示す。

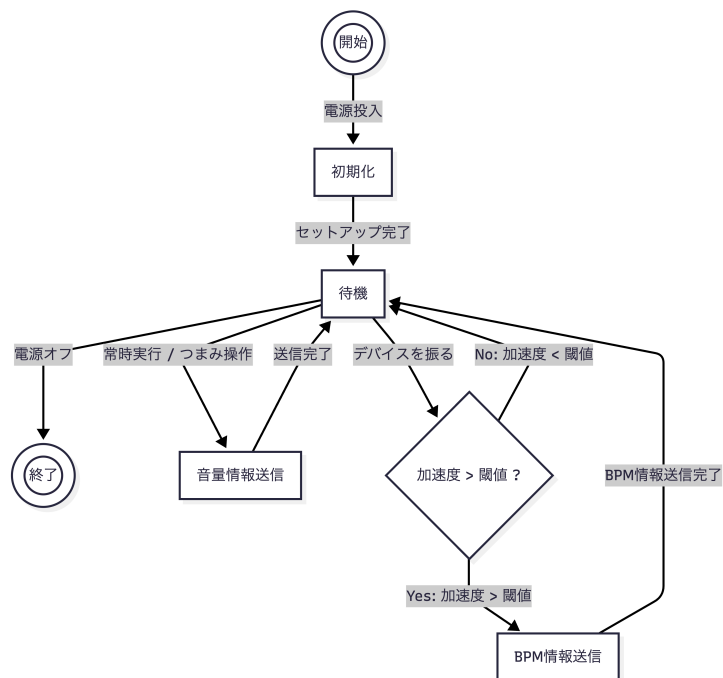


図 2.4 指揮デバイス内部の状態遷移図

第 3 章 システムの詳細設計

楽曲表現の詳細設計について説明を行う。

3.1 部品一覧

楽曲表現を行うにあたり必要となる部品一覧を示す。

3.2 ハードウェア設計

楽曲表現を行う箇所の回路図を図 3.1 に示す。

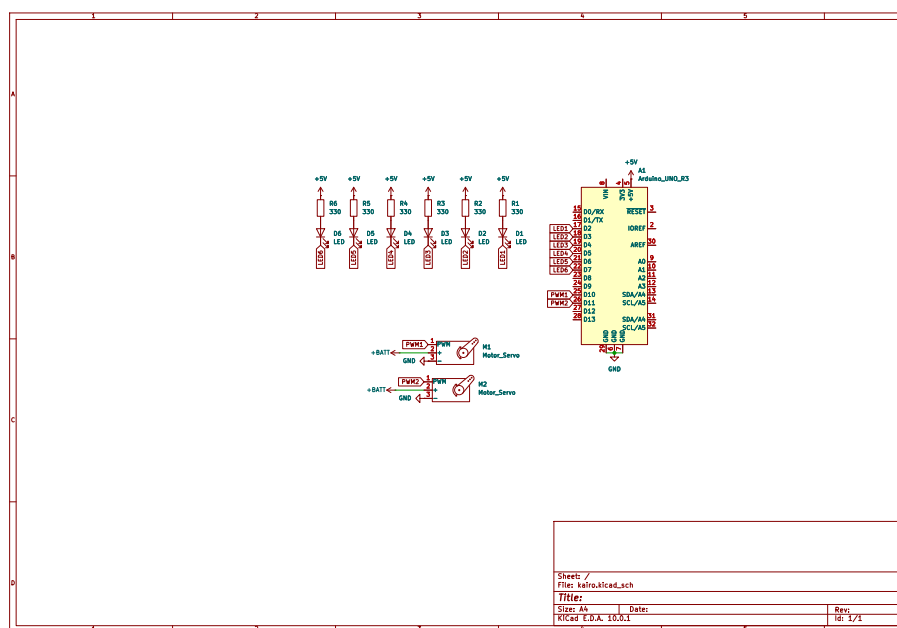


図 3.1 指揮デバイス内部の状態遷移図

3.3 ソフトウェア設計

3.3.1 ファイル構成

Arduino_orchestra/



3.3.2 オブジェクト設計（クラス図）

システムの各モジュールをクラスとして定義し、その構成と関係性を表 3.1 に示す。

依存関係：BpmManager は各コントローラクラス（LedController, ServoDriver, MatrixVisualizer）へタイミング情報および進捗同期データを通知する。

3.3.3 関数設計

各クラスが保持する主要な関数の仕様を表 3.2 に示す。

3.4 処理フローの設計

システム全体のメインループにおける処理フローを図 3.2 に示す。サーボモータは拍の瞬間に関わらず毎ループ `update()` を呼び出すことで、メトロノームのように滑らかに往復動作し続ける。また、LED マトリクスは Arduino 内部に保持するメロディ配列から現在の拍インデックスに対応する音階を取得し、`drawKeyboard()` に渡して表示を更新する。

3.5 テストの設計

表 3.1 オブジェクト設計（クラス定義一覧）

クラス名	種別	内容
BpmManager	属性	bpm : int lastBeatTime : unsigned long
	操作	+ setBpm(bpm : int) : void + isBeatTick() : bool + getProgress() : float
LedController	属性	ledPins[6] : int
	操作	+ begin() : void + blink() : void
ServoDriver	属性	metronomeServo : Servo servoPin : int
	操作	+ begin() : void + update(progress : float) : void
MatrixVisualizer	属性	matrix : ArduinoLEDMatrix
	操作	+ begin() : void + drawKeyboard(note : int) : void + clear() : void

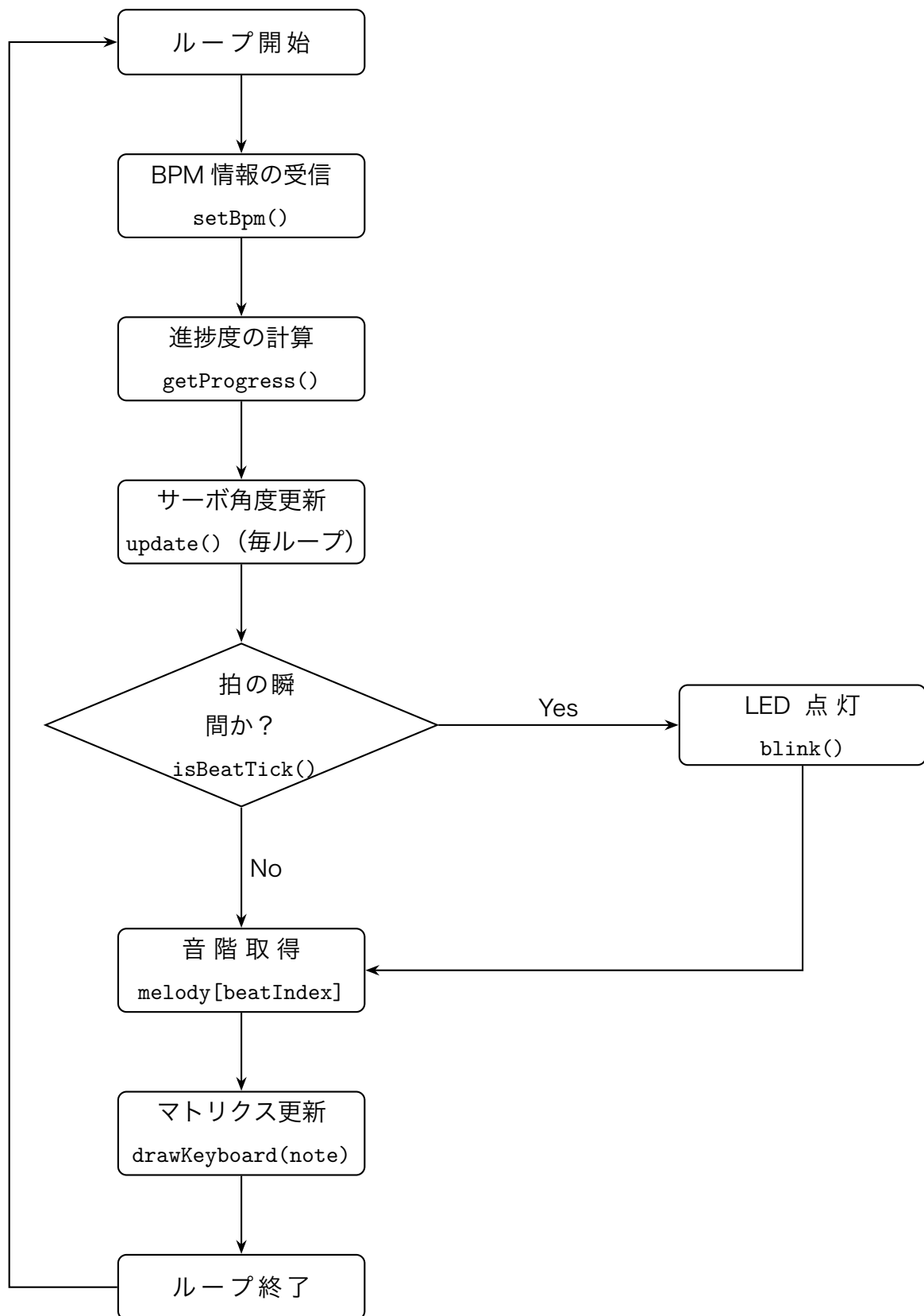


図 3.2 メインループの処理フロー

表 3.2 主要関数仕様一覧

クラス名	関数名	処理内容
BpmManager	setBpm	シリアル受信した BPM 値を内部変数に格納し、拍間隔 (ms) を再計算する.
BpmManager	isBeatTick	現在時刻と前回打点時刻を比較し、拍のタイミングであれば true を返す.
BpmManager	getProgress	現在の拍内における進捗度 (0.0~1.0) を算出し、サーボの滑らかな動きに供する.
LedController	blink	isBeatTick が true の際、全 LED を一定時間点灯させ拍を視覚化する.
ServoDriver	update	進捗度に基づき、サーボをメトロノームのように左右へ往復運動させる.
MatrixVisualizer	drawKeyboard	受信した音階データに対応する LED 列を点灯し、鍵盤のような表示を行う.
MatrixVisualizer	clear	LED マトリクスの全ピクセルを消灯する.